

LLM Uczeń na Raspberry Pi

Benchmark Specification

Cel: znalezienie optymalnego modelu i parametrów generacji
dla lokalnej inferencji CPU-only na Raspberry Pi
w systemie smart-home z odpowiedziami po polsku

1. Kontekst systemu

Wezeł LLM (Uczen) działa na Raspberry Pi jako element systemu asystenta smart-home. Jego zadaniem jest generowanie krótkich odpowiedzi po polsku na podstawie gotowego kontekstu dostarczonego przez większy model (Nauczyciel).

Architektura przepływu danych:

- Frontend wykrywa wake word i nagrywa audio
- Backend wykonuje STT przez Whisper
- Context Service agreguje stan domu przez model Nauczyciel i zwraca system_prompt
- Backend wysyła do Ucznia: system (kontekst domu) + user (pytanie użytkownika)
- Uczeń generuje krótka odpowiedź w języku polskim

Główne wyzwania techniczne:

- Inferencja CPU-only na ARM — każdy wzrost rozmiaru modelu szybko podbija latency
- Mały model dobrze działa tylko przy krótkim, twardym, dobrze uporządkowanym kontekście
- Halucynacje pojawiają się gdy prompt nie wymusza polityki braku danych
- Bottleneck termiczny RPi — po serii zapytań latency rośnie przez throttling

2. Cel benchmarku

Benchmark ma dostarczyć danych empirycznych do odpowiedzi na jedno pytanie:

Jakie połączenie modelu i parametrów generacji daje najlepszy kompromis między jakością odpowiedzi, bezpieczeństwem (brak halucynacji) i latency na konkretnym sprzęcie RPi?

Wynikiem nie jest jeden score — wynikiem są dane i wykresy pozwalające samodzielnie ocenić trade-offy i ostatecznie sformułować własny composite score z wagami dobranymi po analizie.

3. Metryki i uzasadnienie naukowe

Metryki podzielono na trzy grupy według ich charakteru pomiarowego.

3.1 Metryki wydajności (deterministyczne, tylko RPi)

Metryka	Co mierzy	Źródło / uzasadnienie	Jednostka
TTFT	Czas od wysyłki zapytania do pierwszego tokenu	NVIDIA GenAI-Perf; standard branży LLM inference	ms

Metryka	Co mierzy	Zródło / uzasadnienie	Jednostka
E2E Latency	Całkowity czas od zapytania do ostatniego tokenu	Anyscale / NVIDIA benchmarking standards	ms
tokens/s	Przepustowość generacji	LLM-Inference-Bench (arXiv 2411.00136)	tok/s
RAM (RSS)	Zużycie pamięci przez proces Ollama	ELIB edge device benchmark (arXiv 2508.11269)	MB
CPU temp	Temperatura procesora podczas inferencji	Thermal monitoring — specyfika RPi	C
Throttling	Czy RPi ograniczył taktowanie zegara	Thermal stability — stress test	bool

3.2 Metryki jakości (auto, przez Claude Haiku API)

Metryka	Co mierzy	Zródło / uzasadnienie	Skala
Faithfulness	Czy każde twierdzenie w odpowiedzi jest poparte przez kontekst (brak halucynacji faithfulness)	RAGAS — Es et al. 2024 (arXiv 2309.15217); wzór: poparte twierdzenia / wszystkie twierdzenia	0-1
Answer Relevancy	Czy odpowiedź adresuje zadane pytanie, nie inne	RAGAS — Es et al. 2024; DeepEval AnswerRelevancyMetric	0-1
Conciseness	Czy odpowiedź jest zwięzła bez nadmiarowych informacji	ConCISE — Ghafari et al. 2025, Commonwealth Bank (arXiv 2511.16846)	0-1
Consistency	Czy parafrazy tego samego pytania dają tę samą odpowiedź	DefAn — MDPI 2025; RC = zgodne odpowiedzi / wszystkie parafrazy	0-1

3.3 Metryki jakości językowej (auto, przez Claude Haiku API)

Metryka	Co mierzy	Zródło / uzasadnienie	Skala
Polish quality	Naturalność i poprawność języka polskiego — czy odpowiedź brzmi jak naturalny asystent domowy	LLM-as-judge z rubryczkową instrukcją oceny; Haiku ocenia według stałych kryteriów: gramatyka, fleksja, naturalność brzmienia, brak anglicyzmów	0-1
Hallucination flag	Czy odpowiedź zawiera informacje spoza dostarczonego kontekstu	LLM-as-judge — Haiku sprawdza każde twierdzenie pod kątem zakotwiczenia w kontekście; uzupełnienie dla Faithfulness score	bool

Wszystkie 5 metryk jakości liczone są w pełni automatycznie przez Claude Haiku API po każdym request. Brak recenzji człowieka w głównym pipeline. Jeśli wyniki będą wyglądać podejrzanie po pierwszym run, można zrewidować rubryki Haiku i uruchomić benchmark ponownie — koszt ~5 USD za pełny run.

Uwaga: metryki wydajności (TTFT, E2E, tokens/s) mierzone są timestampami po stronie orchestratora (laptop) podczas komunikacji z Ollama na RPi. RAM i temperatura odczytywane z RPi przez /metrics. Metryki jakości liczone na laptopie przez Haiku API — RPi nie jest obciążany żadnym dodatkowym procesem podczas inferencji.

4. Golden dataset

Podstawa ewaluacji jakości jest golden dataset: 20 par {kontekst, pytanie, golden_answer} wygenerowanych przez Claude Opus 4.6 jednorazowo, przed uruchomieniem benchmarku.

4.1 Typy scenariuszy

Typ scenariusza	Opis	Liczba przypadków	Główna metryka
factual_status	Pytanie o aktualny stan (temperatura, oświetlenie, wilgotność)	8	Faithfulness, Conciseness
missing_data	Brak danych w kontekście — model powinien powiedzieć że brakuje	4	Faithfulness (odmowa)
conflicting_data	Sprzeczne dane w kontekście — model powinien wskazać sprzeczność	3	Faithfulness
stale_data	Stary timestamp — model powinien zasignalizować brak pewności	2	Faithfulness
paraphrase	Te same pytania w różnych formach — test Consistency	3	Consistency

4.2 Przykładowa para golden

```
TRYB: brief_status
FAKTY: source=sensor.living_room_temp value=21.4C confidence=0.98
PYTANIE: Jaka jest temperatura w salonie?
GOLDEN ANSWER: W salonie jest teraz 21.4 stopni Celsjusza.
```

5. Przestrzen parametrow

Optuna (Akiba et al., 2019, KDD) przeszukuje przestrzen parametrow za pomoca bayesowskiej optymalizacji — zamiast pelnej siatki testuje ~30 triali na model, kierujac kolejne proby w strone obiecujacych obszarow.

Parametr	Typ	Zakres / Wartosci	Uzasadnienie zakresu
model	categorical	gemma2:2b, llama3.2:3b, phi3.5, qwen2.5:3b, polski wariant	Klasa 2B-4B optymalna dla RPi (context.md)
temperature	float	0.1 - 0.8	Nizsze = bardziej deterministyczne; smart-home preferuje 0.2-0.4 (context.md)
num_ctx	categorical	1024, 2048, 4096	1024-2048 jako optimum dla RPi; 4096 jako kontrprzyklad
num_predict	int	48 - 160	Realne potrzeby odpowiedzi smart-home; powyzej 160 marnuje zasoby
top_p	float	0.80 - 0.99	Standardowy zakres dla RAG-like systemow
repeat_penalty	float	1.0 - 1.15	Lekka kara za powtorzenia — zapobiega loopom

Liczba kombinacji przy pelnej siatce: 5 modeli x 8 wartosci temperatury x 3 ctx x 3 num_predict = 360 kombinacji. Optuna osiaga porownywalne pokrycie w 150 trialach (5 modeli x 30 triali).

6. Pipeline — pelna automatyzacja

Orchestrator (skrypt run_benchmark.py) uruchamiany na laptopie/desktopie. RPi sluzi wylaczenie jako serwer Ollama — zero dodatkowych procesow podczas inferencji. Dzieki temu mierzone metryki wydajnosci odpowiadaja warunkom produkcyjnym.

Komunikacja: laptop wysyla requesty HTTP do Ollama API na RPi (port 11434). Parametry generacji przekazywane per-request w polu options — nie wymaga restartu serwera miedzy trialami Optuny. Monitoring zasobow przez dedykowany lekki endpoint /metrics na RPi (psutil + vcgencmd).

6.1 Schemat dzialania

Dwa komponenty uruchamiane przed benchmarkiem (jednorazowo):

- Na RPi: lekki skrypt metrics_server.py wystawiajacy /metrics (RAM, CPU temp) — jedyny proces poza Ollama
- Na laptopie: wygenerowany golden.jsonl (20 par przez Opus 4.6)

Glowna petla orchestratora (na laptopie):

1. Wczytaj golden.jsonl
2. Dla kazdego modelu:
 - POST /api/pull do RPi — pobierz model jesli nie ma lokalnie
 - Uruchom Optuna study (30 triali)
 - Dla kazdego trialu: wyslij 20 promptow z options {temperature, num_ctx, num_predict, top_p, repeat_penalty}
 - Mierz TTFT i E2E timestampami po stronie laptopa, odpytaj /metrics po kazdym requeście
 - Wyslij odpowiedzi do Claude Haiku API — oblicz wszystkie 5 metryk jakosci
 - Zwroc Optunie composite score, zapisz rekord JSONL
 - DELETE /api/delete — zwolnij RAM RPi przed nastepnym modelem
3. Po wszystkich modelach: wygeneruj report.html z Chart.js

6.2 Format rekordu wynikowego

```
{ "run_id": "2026-04-12_rpi_01", "model": "gemma2:2b", "trial": 14,
  "temperature": 0.3, "num_ctx": 1024, "num_predict": 96,
  "case_id": "status_temp_01", "category": "factual_status",
  "ttft_ms": 820, "e2e_ms": 2140, "tokens_per_s": 21.4,
  "ram_mb": 1840, "cpu_temp_c": 67.2, "throttling": false,
  "faithfulness": 0.95, "answer_relevancy": 0.88,
  "conciseness": 0.91, "consistency": 1.0, "answer": "..."} }
```

7. Ocena jakosci przez Claude Haiku API

Kazda odpowiedz modelu jest oceniana przez Claude Haiku (claude-haiku-4-5) via Anthropic API. Wszystkie 5 metryk jakosci liczone sa w pelni automatycznie — brak kroku recznego przegladu w pipeline. Jesli po analizie wynikow cos wyglada podejrzanie, rubryki Haiku mozna poprawic i uruchomic benchmark od nowa — koszt ~5 USD za pelny run to pomijalny wydatek.

7.1 Koszt ewaluacji

Parametr	Wartosc
Liczba triali (5 modeli x 30)	150 triali
Promptow na trial	20
Laczenie ewaluacji Haiku	~3000
Szacowany koszt Haiku (wszystkie metryki)	~2.50 USD
Calkowity koszt API za pelny run	< 5 USD

7.2 Co ocenia Haiku (wszystkie metryki automatycznie)

- Faithfulness: czy każde twierdzenie w odpowiedzi jest poparte przez dostarczony kontekst
- Answer Relevancy: czy odpowiedź adresuje zadane pytanie, nie inne
- Conciseness: czy odpowiedź nie zawiera nadmiarowych informacji
- Hallucination flag: czy odpowiedź zawiera informacje spoza kontekstu
- Polish quality: naturalność i poprawność języka polskiego — gramatyka, fleksja, brak anglicyzmów, brzmienie asystenta domowego

8. Dashboard i wizualizacja

Statyczny plik `report.html` generowany automatycznie po zakończeniu benchmarku. Nie wymaga serwera — otwierany lokalnie w przeglądarce. Dane wbudowane jako JSON w tagu `script`.

Sekcja dashboardu	Typ wykresu	Co pokazuje
Overview	Tabela rankingowa	Ranking modeli po composite score, zwycięzca per metryka
Latency	Box plot per model	Rozkład TTFT i E2E — średnia, mediana, p95, min/max
Quality	Bar chart per metryka	Faithfulness, Relevancy, Conciseness, Consistency per model
Trade-off	Scatter plot	Latency vs Faithfulness — główny wykres do analizy kompromisu
Parameters	Heatmapa	Temperatura x num_ctx x metryka — widok wpływu parametrów
Thermal	Line chart po czasie	CPU temp i tokens/s w serii 30 requestów — degradacja
Drilldown	Tabela filtrowana	Każdy request: pytanie, odpowiedź, oczekiwana, pass/fail

9. Composite score

Composite score nie jest celem benchmarku — jest narzędziem do porównywania konfiguracji po analizie danych. Wagi dobierane są przez użytkownika po obejrzeniu wyników, nie przed.

Wzór (wagi tymczasowe, do zmiany po analizie):

```
score = w1 * faithfulness + w2 * answer_relevancy + w3 * conciseness
        + w4 * (1 - normalized_latency)
```

Optuna optymalizuje te same wagi wewnętrznie podczas przeszukiwania przestrzeni parametrów. Po benchmarku histogram feature importance z Optuny pokazuje które parametry miały realny wpływ na wynik — to jest dodatkowa, cenna informacja analityczna.

10. Efekty pracy i harmonogram

10.1 Artefakty do wytworzenia

Artefakt	Format	Opis
golden.jsonl	JSONL	20 par testowych wygenerowanych przez Claude Opus 4.6
run_benchmark.py	Python	Orchestrator uruchamiany na laptopie — zarządza modelami na RPi przez Ollama API, zbiera metryki, wywołuje Haiku, generuje raport
metrics_server.py	Python	Lekki serwer na RPi wystawiający /metrics (RAM, CPU temp) — jedyny dodatkowy proces na RPi
results/*.jsonl	JSONL	Surowe dane — jeden rekord per request, wszystkie runy
report.html	HTML + Chart.js	Statyczny dashboard generowany po każdym run
benchmark-spec.md	Markdown	Ta specyfikacja w formacie referencyjnym

10.2 Etapy realizacji

4. Generowanie golden dataset przez Claude Opus 4.6 — 20 par testowych (offline, raz)
5. Implementacja run_benchmark.py z Optuna + Ollama API + Haiku ewaluator
6. Uruchomienie orchestratora na laptopie — ~7 godzin bez nadzoru; RPi tylko serwuje Ollama API
7. Analiza wyników — wykresy trade-off, feature importance z Optuny
8. Dobór wag composite score na podstawie danych
9. Human review próbki (~30 odpowiedzi) — Polish quality score
10. Dokumentacja wyników i rekomendacja konfiguracji produkcyjnej

10.3 Oczekiwany wynik

Konkretna rekomendacja: model X z parametrami Y daje najlepszy kompromis jakości i latencji na RPi dla polskojęzycznego asystenta smart-home. Rekomendacja poparta danymi empirycznymi z ~3000 ewaluacji, odniesieniami do literatury naukowej (RAGAS, ConCISE, DefAn, NVIDIA benchmarking standards) i wizualizacjami trade-offow.